

Estrategia de Optimización de Costos y Recursos

Qué modelo y qué API se usa para cada tarea, y por qué

1. Principio general

La regla del Arquitecto de Flujos es simple: usar el modelo/servicio más barato que resuelva la tarea con la calidad necesaria, y reservar los recursos más caros (modelos grandes, procesamiento en tiempo real) para lo que realmente lo exige — urgencia o complejidad de lectura.

2. Matriz de decisión de modelo por tarea

Tarea	Herramienta elegida	Por qué
Clasificar prioridad + redactar cotización corta (este proyecto)	Gemini 3.1 Flash Lite	Tarea mecánica y de baja complejidad (100-180 palabras, 1 output estructurado). El modelo "lite" es más rápido y hasta 10-15x más barato que un modelo grande, sin pérdida de calidad perceptible para esta tarea.
Lectura densa: análisis de contratos extensos, revisión legal, comparación de cláusulas	Claude (Sonnet)	Tareas de lectura larga y razonamiento contractual se benefician de la ventana de contexto grande y el razonamiento paso a paso de Claude — reduce alucinaciones en análisis de alta dirección (ver Pre-Entrega 6).
Procesar cientos/miles de solicitudes sin urgencia (ej. reclasificar todo el historial de Leads al cierre de temporada)	Message Batches API (Anthropic o equivalente)	No requiere respuesta en segundos. Se agrupan las solicitudes y se procesan en hasta 24 horas con 50% de descuento automático — ahorro certificado sin sacrificar calidad.
Consultas repetidas con el mismo contexto de marca (la base RAG, en cada cotización)	Prompt Caching	El prefijo fijo (rol + instrucciones + tono de marca) se cachea; solo se paga completo por los datos que cambian en cada llamada (el formulario del cliente).

3. Aplicado a este proyecto

El pipeline de Bsquare usa exclusivamente Gemini Flash Lite en tiempo real, porque cada cotización debe redactarse al momento (no es un proceso masivo) y la tarea (clasificar + redactar un texto corto con contexto ya provisto) no exige razonamiento profundo. Si Bsquare decidiera, por ejemplo, reprocesar los 500 Leads históricos para generar un reporte anual de tendencias, esa sí sería una tarea candidata a Batch API (no urgente, volumen alto) combinada con Prompt Caching (el mismo contexto RAG se reutiliza en las 500 llamadas).

4. Estimación de ahorro (proyección a 500 cotizaciones/mes)

Con un prefijo de contexto fijo de ~800 tokens (rol + RAG) y ~1.200 tokens dinámicos por cotización, más ~150 tokens de salida:

Escenario	Costo aproximado (500 cotizaciones)	Ahorro vs. base
Tiempo real, sin caché, sin lote (línea base)	\$2.75	—
Tiempo real + Prompt Caching	\$2.03	26%
Batch API (sin caché)	\$1.38	50%
Batch API + Prompt Caching combinados	\$1.02	63%

Nota: este pipeline corre en tiempo real (cada cotización se procesa al momento que llega, no en lote), por lo que hoy no aplica el descuento de Batch — se documenta como la ruta de optimización disponible si Bsquare decidiera procesar cotizaciones históricas de forma masiva y no urgente.